# ImageLab: long polling

Implementation & comparison with 1s short polling

CMPS 4191 Test 2 | Alex Guerra & Renee Banner

# Short polling: the browser asks on a timer

| Time | Browser | Server |
|---|---|---|
| 0.0 s | GET /v1/jobs/{id} | 200 at once: version 2, processing |
| 1.0 s | GET again (timer) | 200 at once: version 2, unchanged |
| 2.0 s | GET again (timer) | 200 at once: version 4, new progress |
| 3.0 s | GET again (timer) | 200: version 6, completed, so polling stops |

The server never waits. The browser decides when to ask (about 1 s after each response), so a change can sit unseen for up to a second and many requests return nothing new.

# Long polling: the server holds the request

| Time | Browser | Server |
|---|---|---|
| 0.0 s | GET ?after=2&wait=20 | Version is still 2, so it HOLDS the request (no reply) |
| 1.2 s | (waiting) | Worker commits v3, wakes the handler, which rereads and replies 200 v3 |
| 1.2 s | GET ?after=3&wait=20 at once | Holds again |
| 21.2 s | (waiting) | Deadline, nothing new: recheck, then 204 No Content |
| 21.2 s | GET ?after=3 at once | Holds until v6 completed, replies 200, browser stops |

The server answers when something changes, so the browser learns almost immediately.
A 204 is a normal "nothing new yet", not an error.

# Short polling vs long polling

| Polling Events | Short polling | Long polling |
|---|---|---|
| **Who decides when to Poll** | Browser timer (1s) | Server, when something changes |
| **Nothing changed** | Immediate 200 & same version | Held, then 204 at 20s |
| **Time to see a change** | Up to 1s plus poll phase | A few ms after the commit |
| **Requests** | Steady, about 1 per second | 1 per update or per 20s |
| **Server resources** | Short requests, many reads | Held handlers, a notifier, reads on wake |
| **Intermediate versions** | Often skipped | Rarely skipped, more on slow paths |
| **Complexity** | Simple | Versions, waiting, cancellation |

# Overview of Changes Made

## Database

Migration 000004:

- version (starts at 1)
- stage
- variants_completed
- simulation_step

## Data

- Version +1 in the same UPDATE as every change.
  - **Finished jobs can never change again**

## Notifier

- One channel per job:
  - **Channel closing wakes every waiting handler**

## Endpoint

- GET /v1/jobs/{id}?after=N&wait=S:
  - **200, 204, 400 or 404**
- Plain GET unchanged.

## Worker

- Logs commit time
- Then notifies:
  - **progress after each real variant**

## Browser

- Mode selector
- One request at a time
- 204 continuation
- Stop/resume
- Try again

# Backend changes in detail

| Where | What changed |
|---|---|
| migrations/000004 | Adds version (default 1, backfills old jobs), stage, variants_completed, simulation_step, with CHECK constraints |
| data/jobs.go | Every change does version = version + 1 in the same UPDATE; UpdateProgress, UpdateSimulation; WHERE status = processing blocks edits after a terminal state |
| notifier.go (new) | One channel per job; Notify closes it so every waiting handler wakes; Release cleans up |
| api/jobs.go | Parses after and wait (400 on bad values), long-poll handler, no-store, 404 for malformed ids |
| worker.go | After each commit: log commit time, then notify; progress per real variant; optional simulation |
| main.go, server.go | Experiment flags, per-request write deadline, held requests cancelled at shutdown |

# Inside the long-poll handler

## 1 Validate

**Checks**:

- Job id format
- Then after:
  - integer: 0 or more
- &wait:
  - is 1 to 20

## 2 Subscribe

- Registers with the notifier
- Captures the current change channel

## 3 Read

- Loads the job from PostgreSQL

## 4 Decide

- Terminal or newer:
  - 200 now.
  - after too high: 400
  - Otherwise hold

## 5 Wait

- Block on the channel
- The 20s timer
- Or the client disconnects

## 6 Finish

- Woken:
  - reread
- Deadline:
  - recheck then 204.

**Always release and stop the timer**

# Browser changes in detail

- Start: one prompt GET saves the baseline version, then the chosen mode runs
- Long polling api sends ?after=lastVersion&wait=20 with a 30 s timeout
- 200 with a newer version: render and ask again immediately
- 204: keep the version, do not parse JSON, ask again immediately
- Completed or failed: render and stop
- Mode switch or Stop: abort the request; a generation token ignores late responses
- Network error, timeout or bad JSON: keep last state (same job, no new upload)

# How a held request avoids missing a change

| Situation | Response |
| --- | --- |
| Client behind, or job finished | 200 immediately |
| Versions match, job active | Hold up to 20 s |
| Deadline, nothing new | 204, no body |
| after above current version | 400 immediately |
| Unknown or malformed id | 404 |

1. Handler subscribes and captures the change channel

2. Handler then reads the job from PostgreSQL

3. Worker commits, logs the time, then closes the channel

* **A signal only means "read again".**
**It never proves success.**

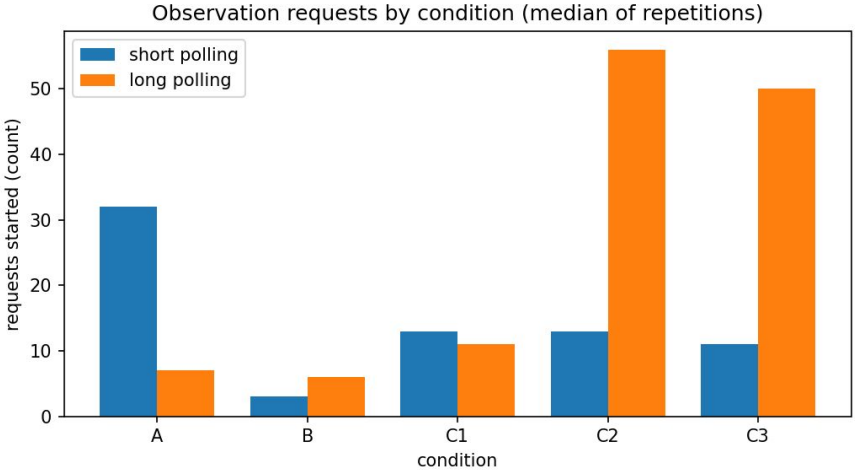
# Experiment design: 31 runs

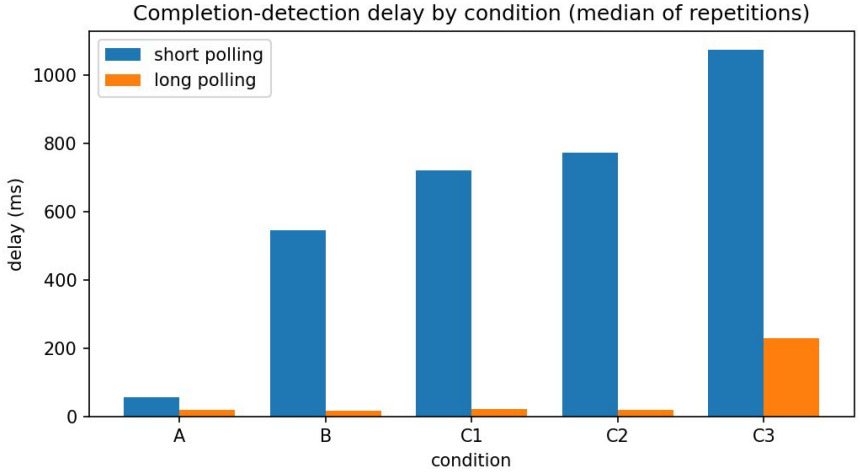
| Condition | Workload |
|---|---|
| A quiet | 30s delay, no updates |
| B real progress | three real variants |
| C1 slow updates | simulated update every 2s for 10s |
| C2 fast updates | simulated update every 200ms for 10s |
| C3 fast + latency | C2 plus 200ms (DevTools) |

**Controls**

- Same image, empty queue, one foreground tab
- 3 runs per condition
- Delay = browser receipt minus server commit time, same computer

# Results: requests and detection delay




Long polling learned the result sooner in every condition, but sent fewer requests only when the job was quiet (A).

# Data Observations: Long Polling vs. Short Polling

**LP vs SP**
**15 ms vs 545 ms**

Detection delay in B (real progress)

**SP vs LP**
**32 vs 7**

Requests in A: 25 empty requests avoided in long polling

**SP vs LP**
**13 vs 56**

Requests in C2: long polling sends one per update

**0 vs 5-6**

Versions missed by long polling in C2 vs C3

**Processing time did not change. Only the time to discover the result did.**
**Fewer requests can still mean more database reads (C2: 109 vs 13).**

# Take Away

- Ordinary use: long polling is sufficient and clearly better on delay
- Short polling is fine for very short jobs or on simple servers
- Fast updates over a slow path: Version skipping for long polling occurs
- Next design: one continuing response sending several updates
- Limits: 3 repetitions, one machine, short-poll phase clustered, job-table reads only